Software Requirements Specification

Multi-Tenant Ticketing & Booking Platform

FairPath Backend · NestJS 12 · Prisma 7 · PostgreSQL

Version:	**Date:**	**Status:**	**DB Schema:**
v2.0	11 November 2026	Active	v2.5 — 86 Tables · 173 Relationships

Table of Contents

1. 1. Introduction & Purpose
2. 2. Scope & Product Overview
3. 3. Stakeholders & User Roles
4. 4. Architecture & Technical Stack
5. 5. Functional Requirements by Cluster
    1. 5.1 Cluster A — Tenancy & Access
    2. 5.2 Cluster B — Customers & Integration
    3. 5.3 Cluster C — Registries & Rules
    4. 5.4 Cluster D — Catalog
    5. 5.5 Cluster E — Operations & Allocation
    6. 5.6 Cluster F — Booking & Money
    7. 5.7 Cluster G — Supplier Integration
    8. 5.8 Cluster H — Crew Management
6. 6. Database Architecture (86 Tables)
7. 7. Non-Functional Requirements
8. 8. Security & Compliance
9. 9. API Design Standards
10. 10. Implementation Status
11. 11. Revision History

1. Introduction & Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the **Multi-Tenant Ticketing & Booking Platform** (codename: **FairPath**). The platform is designed to serve as an enterprise-grade, white-label booking infrastructure supporting multiple transport operators (tenants) simultaneously, with unified inventory management across all sales channels.

This document is authoritative for all development, QA, and integration activities. It is derived from the original SRS v1.0 and has been updated to reflect the finalized database schema v2.5

(86 tables) and the API specification v1.0 (78 operations across 8 functional clusters).

> **Scope of Update (v2.0):** This revision incorporates the complete Prisma schema (86 models), the full 8-cluster API surface, KYC submission workflow, the `ResourceDocument` and `ResourcePart` models, the Workflow engine models (`Workflow` , `WorkflowStatus` , `WorkflowTransition`), loyalty accounts, promo redemptions, and the Platform Finance settlement approval flow.

2. Scope & Product Overview

FairPath is a **multi-tenant SaaS platform** enabling transport operators (bus, ferry, train, event) to sell tickets across five concurrent channels without inventory conflicts:

Channel	Code	Description
Marketplace	`MARKETPLACE`	Platform-owned OTA consumer portal
Tenant Web	`TENANT_WEB`	Operator's branded white-label website
Counter	`COUNTER`	Physical ticket counter POS application
Agent	`AGENT`	Third-party travel agency portal
API	`API`	Server-to-server integrations via API key

Key Architectural Tenets

1. **Multi-Tenancy & Isolation** — Full logical isolation by `tenant_id` enforced at application layer and PostgreSQL Row-Level Security (RLS).
2. **Channel-Agnostic Inventory Engine** — Unified shared inventory accessed simultaneously by all channels with per-channel allocation pools.
3. **Double-Entry Ledger Accounting** — Append-only transactional financial ledger for zero reconciliation leakage across online wallets, cash counters, and payment gateways.
4. **Resilient Lock & Hold Pipeline** — Atomic distributed hold reservations preventing double-booking under extreme peak concurrency (e.g. Eid holiday rushes).
5. **Maker-Checker Governance** — Strict four-eyes dual-authorization for sensitive financial overrides, fare revisions, and quota adjustments inside cutoff windows.
6. **Extensible Attribute System** — Runtime attribute schema definitions without database migrations via `AttributeDefinition` + JSON columns.
7. **Event-Driven Architecture** — Transactional outbox pattern (`OutboxEvent`) guarantees at-least-once delivery of domain events to downstream consumers.
8. **Workflow Engine** — Generic workflow state machine (`Workflow` , `WorkflowStatus` , `WorkflowTransition`) for KYC, approval, and onboarding flows.

3. Stakeholders & User Roles

Role	Scope	Key Capabilities
Platform Super Admin	Cross-tenant	Tenant CRUD, plan management, platform-wide audit, settlement approvals, platform finance operations
Tenant Admin	Own tenant	Staff management, branch setup, fare configuration, resource registration, schedule management
Operations Manager	Own tenant	Trip management, crew assignment, seat allocation, vehicle swaps, emergency crew replacement
Counter Staff	Own branch	Counter ticket sales, cash payments, seat selection, booking cancellations
Finance Checker	Own tenant	Approve/reject maker-submitted sensitive operations (four-eyes principle)
Agent	Contracted	Book on behalf of customers using credit wallet, view agent-pool inventory
Customer	Self-service	OTP login, seat search, hold, payment, ticket download, cancellation
Crew Member	Assigned trips	View assigned trips, check in, acknowledge duty

Authentication Methods

- **Staff & Agents**: Email/Password + optional TOTP MFA → JWT Bearer token
- **Customers**: Phone OTP → short-lived JWT
- **Server-to-server**: API Key pair (`X-API-Key` header)
- **External SSO**: Federated identity support (`StaffFederatedIdentity` , `CustomerFederatedIdentity`)

4. Architecture & Technical Stack

Layer	Technology	Notes
Runtime	Node.js 22 LTS	ESM-only modules, `tsx` for TS execution
Framework	NestJS 12	ESM, platform-node adapter, `NodeAdapter`
ORM	Prisma 7	pg driver adapter, BigInt IDs, generated client
Database	PostgreSQL 16	RLS, JSONB, Timestamptz, partial indexes
Observability	@nestjs/observe	Traces, errors, logs, metrics via Observe platform
Auth	JWT + bcryptjs	MFA via TOTP, federated identity support
Validation	StandardSchema / class-validator	Zod or ArkType compatible
Testing	Vitest 4	`vi.fn()` mocks, ESM-compatible
Linting	oxlint + tsgolint	TypeScript-aware

> **ESM-First:** All relative imports end in `.js`. Use `import.meta.dirname` instead of `__dirname`. No CommonJS require() calls.

5. Functional Requirements by Cluster

Cluster A — Tenancy & Access (17 API Operations)

Tenant onboarding, subscription plans, staff RBAC, and maker-checker approval pipelines.

FR-A1: Tenant Lifecycle Management

- Platform Super Admin can create, read, update tenants through states: `APPLIED → KYC_REVIEW → ACTIVE → SUSPENDED → TERMINATED`
- Each tenant has a **subscription plan** (`Plan`) controlling feature flags and channel limits
- Payment model: `PLATFORM_COLLECT` (platform collects, settles to tenant) or `TENANT_COLLECT` (tenant collects directly)
- Tenants can configure custom domain, branding, and product types (BUS, TRAIN, FERRY, EVENT)

FR-A2: KYC Submission Workflow

- New tenants submit KYC documents via `KycSubmission` model
- Platform admin reviews and transitions tenant to `ACTIVE` status
- Workflow state machine tracks review steps via `Workflow` , `WorkflowStatus` , `WorkflowTransition`

FR-A3: Branch & Counter Management

- Tenants can register multiple physical branches/counters with city, address, phone
- Branch codes follow format: `{CITY}-{LOCATION}-{SEQ}` (e.g. `DHK-GAB-01`)

FR-A4: Agent Contracting

- Tenants can onboard travel agents with credit wallet limits
- Agent commission rates and credit balances tracked per tenant

FR-A5: Staff RBAC

- Staff users assigned to tenants via `Membership` (many-to-many)
- Role-based permissions via `Role` → `RolePermission` → `Permission`
- MFA support via TOTP (`mfa_enabled` flag on `StaffUser`)
- Federated SSO via `StaffFederatedIdentity`
- Sessions tracked in `StaffSession`

FR-A6: Maker-Checker Approval Pipeline

- Sensitive operations (fare changes, quota adjustments, refund overrides) require a second approver
- Maker submits `ApprovalRequest` with action payload; checker cannot be the same user
- States: `PENDING → APPROVED | REJECTED`

FR-A7: Immutable Audit Trail

- All state-changing operations append to `AuditLog` (append-only, never updated)
- Queryable by tenant, action type, actor, date range

API Endpoints

POST	**/auth/login**	Staff authentication — email/password + optional MFA code → JWT
GET	**/tenants**	List all tenants (Platform Super Admin)
POST	**/tenants**	Create/onboard new tenant operator
GET	**/tenants/{id}**	Get tenant details
PUT	**/tenants/{id}**	Update tenant profile or status
GET	**/tenants/{tenantId}/branches**	List tenant counter branches
POST	**/tenants/{tenantId}/branches**	Create branch counter
GET	**/tenants/{tenantId}/agents**	List contracted agents
POST	**/tenants/{tenantId}/agents**	

Onboard and register agent

GET	**/plans**	List subscription plans
GET	**/staff-users**	List staff user accounts
POST	**/staff-users**	Create staff user account
GET	**/approval-requests**	List maker-checker approval requests
POST	**/approval-requests**	Submit sensitive action for checker approval
POST	**/approval-requests/{id}/approve**	Checker approves pending request (enforces maker ≠ checker)
POST	**/approval-requests/{id}/reject**	Checker rejects pending request
GET	**/audit-logs**	Query immutable append-only audit trail

DB Models

`Tenant` `TenantProductType` `TenantSetting` `Branch` `Agent` `Plan` `PlanFeature` `Feature` `StaffUser` `StaffFederatedIdentity` `StaffSession` `Membership` `Role` `Permission` `RolePermission` `ApprovalRequest` `AuditLog` `KycSubmission` `Workflow` `WorkflowStatus` `WorkflowTransition`

Cluster B — Customers & Integration (10 API Operations)

Customer OTP authentication, profile management, saved travelers, API keys, and webhook subscriptions.

FR-B1: Customer Authentication

- Customers authenticate via phone OTP (2-step: send → verify)
- OTP validity window: 5 minutes; maximum 3 attempts before lockout
- Successful verification issues a short-lived JWT

- Federated identity support via `CustomerFederatedIdentity` (Google, Facebook, Apple)
- Sessions tracked in `CustomerSession`

FR-B2: Customer Profile

- Customers have per-tenant profiles (`CustomerProfile`) with marketing consent flags
- Linked to base `CustomerIdentity` (phone-based, cross-tenant)
- GDPR-compliant consent fields: `marketing_opt_in`, `data_sharing_consent`

FR-B3: Saved Travelers

- Customers can save companion traveler profiles for fast checkout
- Each saved traveler can have identity documents (`SavedTravelerDocument`)

FR-B4: API Key Management

- Tenants generate API key pairs for server-to-server integrations
- Keys scoped to tenant; revocable; hashed storage

FR-B5: Webhook Subscriptions

- Tenants register webhook endpoints (`WebhookEndpoint`) for domain events
- Events delivered via transactional outbox (`OutboxEvent`) guaranteeing at-least-once delivery
- Idempotency keys (`IdempotencyKey`) prevent duplicate processing

API Endpoints

POST	**/customer/otp/send**	Send login or checkout OTP to phone number
POST	**/customer/otp/verify**	Verify OTP and issue customer access token
GET	**/customers/me**	Get current customer profile & settings
PUT	**/customers/me**	Update customer marketing and privacy consent preferences
GET	**/customers/saved-travelers**	List saved travelers for current profile
POST	**/customers/saved-travelers**	

Save companion traveler for fast checkout

GET	**/api-keys**	List registered tenant API keys
POST	**/api-keys**	Generate new API key pair
GET	**/webhooks**	List configured webhook subscriber endpoints
POST	**/webhooks**	Register new webhook destination URL

DB Models

`CustomerIdentity` `CustomerFederatedIdentity` `CustomerProfile` `CustomerSession` `SavedTraveler` `SavedTravelerDocument` `SalesDomain` `ApiKey` `WebhookEndpoint` `OutboxEvent` `IdempotencyKey`

Cluster C — Registries & Rules (8 API Operations)

Product type registry, dynamic attribute definitions, business rules engine, and currency/exchange rates.

FR-C1: Product Type Registry

- Platform maintains a registry of product types: `BUS`, `TRAIN`, `FERRY`, `EVENT`
- Tenants subscribe to product types they operate

FR-C2: Lookup Types & Values

- Configurable enum registries (`LookupType` → `LookupValue`) for dynamic dropdown data
- Examples: seat position codes, cell type codes, unit type codes, amenity tags

FR-C3: Dynamic Attribute Schema

- `AttributeDefinition` allows tenants to define custom attributes on entities without schema migration
- Attribute values stored in JSONB columns on associated entities

- Supports types: `TEXT`, `NUMBER`, `BOOLEAN`, `DATE`, `ENUM`

FR-C4: Business Rules Engine

- `Rule` model stores versioned business rules: price modifiers, refund policies, eligibility criteria, quota adjustments
- Rules have priority ordering and condition expressions evaluated at runtime
- Rule categories: `PRICE`, `REFUND`, `ELIGIBILITY`, `QUOTA`

FR-C5: Currency & Exchange Rates

- Platform maintains a currency registry (`Currency`) and active exchange rates (`ExchangeRate`)
- All monetary values stored in minor units (e.g. 150000 = 1500.00 BDT)
- ISO 4217 currency codes

API Endpoints

GET	**/product-types**	List supported product types (BUS, TRAIN, FERRY, EVENT)
GET	**/lookup-types/{typeCode}/values**	Get lookup values for a registry type
GET	**/attribute-definitions**	List dynamic attribute schema definitions
POST	**/attribute-definitions**	Define new extensible attribute schema without DB migration
GET	**/rules**	List business rules (price, refund, eligibility, quota)
POST	**/rules**	Create or version business rule
GET	**/currencies**	List platform currencies
GET	**/exchange-rates**	List active exchange conversion rates

DB Models

`ProductType` `LookupType` `LookupValue` `AttributeDefinition` `PolicyDefinition` `Rule`

`Template` `Currency` `ExchangeRate`

Cluster D — Catalog (15 API Operations)

Locations, vehicle offerings, seat layout templates, fleet resources, routes, schedules, fare rules, and ancillaries.

FR-D1: Location Registry

- Hierarchical location registry: city → terminal → boarding point
- Searchable by name, type, geographic proximity

FR-D2: Offerings (Service Classes)

- `Offering` defines a commercial tier (e.g. "Double Deck Luxury AC Sleeper")
- Linked to `ProductType` and optional features (`OfferingFeature`)
- Seat selection modes: `MANDATORY` , `AUTO_ASSIGN` , `NONE`

FR-D3: Seat Layout Templates

- `LayoutTemplate` defines the grid structure of a vehicle's seating layout
- `LayoutCell` defines each cell: seat/aisle/lavatory, row/col position, tier, flags (ACCESSIBLE, PREMIUM_LEG_ROOM), sell mode
- Layout publication requires maker-checker approval
- Status flow: `DRAFT → REVIEW → PUBLISHED → ARCHIVED`

FR-D4: Fleet Resources

- `Resource` represents a physical vehicle (bus, coach, train car)
- Linked to `Offering` and `LayoutTemplate`
- Supports multi-part vehicles via `ResourcePart`
- Document storage for maintenance records via `ResourceDocument`
- Status: `ACTIVE` , `MAINTENANCE` , `RETIRED`
- Crew permanently assigned to resources via `ResourceCrew`

FR-D5: Service Lines (Routes)

- `ServiceLine` defines an ordered sequence of stops with time offsets from origin
- `ServiceStop` records: sequence, location, offset minutes, pickup/drop flag

FR-D6: Schedules

- `Schedule` defines recurring departure patterns: days of week, departure time, validity period
- Linked to service line and optional default resource assignment
- Trip instances auto-generated from schedules by the scheduling engine

FR-D7: Fare Rules

- Composable pricing via multiple `FareRule` components per offering/route
- Component types: `BASE`, `SEAT_TIER`, `MODIFIER`, `DYNAMIC`, `TAX`
- Amounts stored in minor currency units
- Time-bound validity with maker-checker for changes inside cutoff

FR-D8: Ancillary Products

- `AncillaryProduct`: add-ons like extra baggage, meals, travel insurance
- `AncillaryOffer`: trip-specific pricing of ancillary products
- Categories: `BAGGAGE`, `MEAL`, `INSURANCE`, `COMFORT`

API Endpoints

GET	**/locations**	Search cities, terminals, and boarding points
GET	**/offerings**	List vehicle/service classes & tiers
POST	**/offerings**	Create new commercial offering tier
GET	**/layout-templates**	List seat layout map templates
POST	**/layout-templates**	Design new seat layout template grid
POST	**/layout-templates/{id}/publish**	Publish layout template (submits for maker-checker review)

GET	**/resources**	List physical fleet resources (buses, coaches)
POST	**/resources**	Register vehicle to tenant fleet
GET	**/service-lines**	List routes and service lines
POST	**/service-lines**	Define service line with ordered stops and time offsets
GET	**/schedules**	List recurring departure timetables
POST	**/schedules**	Create recurring timetable schedule
GET	**/fare-rules**	List fare rules and pricing components
POST	**/fare-rules**	Configure fare component (BASE, SEAT_TIER, etc.)
GET	**/ancillaries**	List add-on products and meal/baggage offers

DB Models

`Location` `Offering` `OfferingFeature` `LayoutTemplate` `LayoutCell` `Resource` `ResourcePart` `ResourceDocument` `ServiceLine` `ServiceStop` `Schedule` `FareRule` `AncillaryProduct` `AncillaryOffer`

Cluster E — Operations & Allocation (6 API Operations)

Trip generation, seat inventory snapshots, atomic seat holds, and pool allocation quotas.

FR-E1: Trip Management

- `Trip` is a concrete departure instance generated from a `Schedule`
- Trip contains frozen snapshots: `layout_snapshot`, `offering_snapshot`, `policy_version`
- Supply modes: `PLATFORM_OWNED`, `SUPPLIER_FEED`
- Status flow: `SCHEDULED → OPEN → DEPARTED → COMPLETED | CANCELLED`

- Trip stops materialized in `TripStop` for real-time ETD/ETA tracking
- Trip changes (delays, cancellations) logged in `TripChange`

FR-E2: Seat Inventory (TripUnit)

- Each seat in a trip is a `TripUnit` with real-time status: `AVAILABLE`, `HELD`, `BOOKED`, `BLOCKED`
- Units partitioned into channel-specific allocation pools visible to corresponding channels only
- Multi-berth units (sleeper cabins) linked via `UnitSpan`

FR-E3: Seat Hold (Atomic Locking)

- Atomic distributed hold via database row-level locking — no double-booking possible
- Hold auto-expires (default: 10 minutes) if payment not completed
- `Hold` → `HoldItem` (one per seat/unit held)
- Manual early release supported

FR-E4: Allocation Rules

- `AllocationRule` defines channel quota splits (e.g. 60% MARKETPLACE, 30% COUNTER, 10% AGENT)
- Allocation events (`AllocationEvent`) track all pool movements
- Quota adjustments require maker-checker approval inside cutoff windows

FR-E5: Emergency Vehicle Swap

- Operations managers can swap the resource on a live trip
- System re-maps existing bookings to new layout (best-effort seat preservation)
- Affected customers notified via event pipeline

API Endpoints

GET	**`/trips/search`**	Search available trips across channels with cached availability
GET	**`/trips/{id}/seatmap`**	Get live seat map with channel-specific unit pool visibility and prices
POST	**`/trips/{id}/swap-resource`**	Perform emergency vehicle swap on a scheduled trip

POST	**/holds**	Create atomic seat hold reservation with auto-expiration timer
DELETE	**/holds/{id}/release**	Manually release held seats back to available
POST	**/allocation/move-units**	Move seats between pools (e.g. counter to shared)

DB Models

`Trip` `TripStop` `TripUnit` `UnitSpan` `Hold` `HoldItem` `AllocationRule` `AllocationEvent` `TripChange`

Cluster F — Booking & Money (10 API Operations)

Booking creation, traveler management, payment processing, ticket issuance, cancellation, refunds, and settlement.

FR-F1: Booking Creation

- Booking converts an active `Hold` into a confirmed `Booking`
- Each booking has: travelers (`Traveler`), items (`BookingItem`), price breakdown (`BookingPriceLine`), and issued tickets (`Ticket`)
- Booking reference: human-readable unique code (e.g. `BK-20261109-9X7F`)
- Traveler documents stored in `TravelerDocument`

FR-F2: Payment Processing

- Supports multiple payment methods: online gateway, mobile wallet, cash counter, agent credit
- Payment session initiated via `/payments/initiate` → gateway redirect
- Gateway callback via signed server-to-server webhook (`/payments/webhook`)
- All payment events recorded in `Payment` model
- Double-entry ledger entries in `LedgerEntry`

FR-F3: Wallet & Loyalty

- `WalletAccount`: tenant-scoped wallet balances for agents and customers
- `LoyaltyAccount`: points accumulation and redemption
- `Promo` + `PromoRedemption`: promotional discount codes with usage limits

FR-F4: Ticket Issuance & Validation

- Tickets issued as QR-encoded payloads after payment confirmation
- Online and offline QR validation at gate (`/tickets/{ticketNo}/verify`)
- Boarding scan records passenger boarding time (`/tickets/{ticketNo}/board`)

FR-F5: Cancellation & Refunds

- Dynamic refund computation based on time-to-departure slabs (via `Rule` engine)
- Refund records in `Refund` model, settled via original payment method or wallet

FR-F6: Settlement

- Periodic settlement cycles for tenant payouts (`Settlement` model)
- Platform Finance maker-checker approval required before disbursement

API Endpoints

POST	**/bookings**	Create booking from active hold with travelers and breakdown
GET	**/bookings**	List customer or tenant bookings
GET	**/bookings/{id}**	Retrieve complete booking ledger, items, and issued tickets
POST	**/payments/initiate**	Initiate payment session with gateway or wallet
POST	**/payments/webhook**	Payment gateway signed server-to-server callback
GET	**/tickets/{ticketNo}/verify**	Validate QR ticket payload offline or online
POST	**/tickets/{ticketNo}/board**	Scan ticket at gate / bus door to mark boarded
POST	**/bookings/{id}/cancel**	Cancel booking and compute refund against dynamic time slabs
GET	**/settlements**	

List periodic tenant settlement cycles and statements

POST **/settlements/{id}/approve**
Platform Finance maker-checker approval for tenant payout disbursement

DB Models

`Booking` `Traveler` `TravelerDocument` `BookingItem` `BookingPriceLine` `Ticket` `Payment` `Refund` `Promo` `PromoRedemption` `WalletAccount` `LoyaltyAccount` `LedgerEntry` `Settlement` `NotificationLog`

Cluster G — Supplier Integration (3 API Operations)

Third-party CRS/GDS supplier adapters, external ID cross-mapping, and integration log audit trail.

FR-G1: Supplier Connections

- `SupplierConnection` configures external supply adapter (CRS, GDS, aggregator)
- Supports multiple adapter types with configurable credentials and endpoint config
- Connection status tracking: `ACTIVE`, `PAUSED`, `ERROR`

FR-G2: External ID Mappings

- `ExternalMapping` cross-maps internal entity IDs to supplier external codes
- Entity types: location, service_line, resource, offering, trip

FR-G3: Supplier Booking References

- `SupplierBookingRef` links internal bookings to supplier confirmation references
- `IntegrationLog` tracks all supplier API calls for debugging and reconciliation

API Endpoints

GET **/supplier-connections**
List third-party CRS/GDS supplier connections

POST **/supplier-connections**
Configure external supply adapter connection

GET **/external-mappings**
List entity ID cross-mappings with external supplier

DB Models

`SupplierConnection` `ExternalMapping` `SupplierBookingRef` `IntegrationLog`

Cluster H — Crew Management (9 API Operations)

Crew member records, statutory license compliance, rest rule enforcement, leave management, and duty rosters.

FR-H1: Crew Member Registry

- `CrewMember` stores driver/helper/contractor profiles linked to optional `StaffUser`
- Employment types: `EMPLOYEE`, `CONTRACT`, `DAILY`
- Status: `ACTIVE`, `ON_LEAVE`, `SUSPENDED`, `LEFT`
- Crew roles defined per offering/product type via `CrewMemberRole`
- Minimum crew requirements per route/offering via `CrewRequirement`

FR-H2: Statutory Document Compliance

- Mandatory documents tracked in `CrewDocument`: `DRIVING_LICENSE`, `NID`, `HEALTH_CERT`, `TRAINING_CERT`
- License expiry validation blocks trip assignment
- Document numbers stored encrypted; physical verification flag tracked

FR-H3: Leave & Rest Management

- `CrewLeave` records approved/pending leaves: `ANNUAL`, `MEDICAL`, `EMERGENCY`, `REST_DAY`
- Approved leaves block crew assignment during the date range
- Minimum rest hours between shifts enforced by assignment engine

FR-H4: Trip Crew Roster

- `TripCrew` records crew assigned to trips with role, duty status, timestamps
- Assignment validates: license validity, leave conflicts, minimum rest, concurrent trip overlap
- Duty states: `PENDING`, `CONFIRMED`, `CHECKED_IN`, `REPLACED`

FR-H5: Emergency Crew Replacement

- Operations managers can replace a crew member on a live trip
- Replacement linked via `replaced_by_id` foreign key for audit trail
- Replacement triggers license and availability checks on the substitute

FR-H6: Resource Crew Assignments

- `ResourceCrew` allows permanent crew assignments to vehicles within validity periods

API Endpoints

GET	**/crew-members**	List crew members (drivers, contractors, helpers)
POST	**/crew-members**	Register crew member profile and role assignments
GET	**/crew-members/{id}/documents**	List crew statutory licenses and health certificates
POST	**/crew-members/{id}/documents**	Upload and verify driving license or statutory document
GET	**/crew-members/{id}/leaves**	List crew leave and rest schedules
POST	**/crew-members/{id}/leaves**	Submit or approve crew leave request (blocks trip assignment during dates)
GET	**/trips/{id}/crew**	List assigned crew roster for a trip
POST	**/trips/{id}/crew**	Assign crew member to a trip role (validates license, rest rules, overlap lock)
POST	**/trips/{id}/crew/{tripCrewId}/replace**	Emergency replacement / swap of sick or absent crew member

DB Models

`CrewMember` `CrewMemberRole` `CrewDocument` `CrewLeave` `CrewRequirement` `ResourceCrew` `TripCrew`

6. Database Architecture (86 Tables)

The complete schema is defined in `prisma/schema.prisma` (Prisma 7, PostgreSQL 16). All tables use `BigInt` auto-increment primary keys, `Timestamptz(6)` for timestamps, and snake_case column names via `@@map`.

Cluster	Table Count	Key Entities
A — Tenancy & Access	21	tenant, branch, agent, plan, staff_user, role, permission, approval_request, audit_log, kyc_submission, workflow*
B — Customers & Integration	11	customer_identity, customer_profile, saved_traveler, api_key, webhook_endpoint, outbox_event, idempotency_key
C — Registries & Rules	9	product_type, lookup_type, lookup_value, attribute_definition, rule, policy_definition, template, currency, exchange_rate
D — Catalog	14	location, offering, layout_template, layout_cell, resource, resource_part, resource_document, service_line, service_stop, schedule, fare_rule, ancillary_product
E — Operations	9	trip, trip_stop, trip_unit, unit_span, hold, hold_item, allocation_rule, allocation_event, trip_change
F — Booking & Money	15	booking, traveler, traveler_document, booking_item, booking_price_line, ticket, payment, refund, promo, promo_redemption, wallet_account, loyalty_account, ledger_entry, settlement, notification_log
G — Supplier Integration	4	supplier_connection, external_mapping, supplier_booking_ref, integration_log
H — Crew Management	7	crew_member, crew_member_role, crew_document, crew_leave, crew_requirement, resource_crew, trip_crew

Total: **90 tables** (86 domain + 4 generated junction/pivot models incl. feature/plan relationships).

> **New in v2.5:** `ResourcePart`, `ResourceDocument`, `KycSubmission`, `Workflow`, `WorkflowStatus`, `WorkflowTransition`, `LoyaltyAccount`, `PromoRedemption`, `SupplierBookingRef`, `CrewMemberRole`,

> `CrewRequirement`, `ResourceCrew` added since v1.0.

7. Non-Functional Requirements

Category	Requirement	Target
Availability	Production uptime SLA	99.9% monthly
Performance	Trip search P95 latency	< 200ms (cached)
Performance	Seat hold creation P95	< 500ms
Performance	Payment webhook processing	< 1s
Concurrency	Peak concurrent holds (Eid)	500+ simultaneous, zero double-booking
Data Integrity	Financial ledger consistency	Double-entry, zero reconciliation leakage
Scalability	Horizontal API scaling	Stateless, Kubernetes-ready
Observability	Distributed tracing	100% trace coverage via @nestjs/observe
Reliability	Event delivery guarantee	At-least-once via transactional outbox
Reliability	Idempotency	All mutating endpoints support idempotency keys

8. Security & Compliance

- **Multi-Tenant Isolation**: PostgreSQL Row-Level Security on all tenant-scoped tables; application-layer `tenant_id` enforcement
- **Authentication**: JWT with 1-hour expiry; refresh token rotation; MFA via TOTP
- **Encryption at Rest**: Sensitive fields (NID, license numbers, card data) encrypted before storage (`_encrypted` suffix)
- **Audit Trail**: Immutable append-only `AuditLog` ; no UPDATE/DELETE permitted on audit records
- **Payment Security**: Webhook signature validation (HMAC-SHA256); PCI-DSS compliant payment flow (no card data stored)
- **Data Privacy**: GDPR-compliant consent management; customer data portability; right to erasure via anonymization
- **API Keys**: Keys hashed (bcrypt); transmitted once; revocable immediately
- **Maker-Checker**: Four-eyes principle enforced at API layer; same-user approval rejected with 403
- **Rate Limiting**: OTP endpoints: 3 attempts / 5-minute window; API key: configurable per-tenant

9. API Design Standards

- **Base URL**: `https://api.platform.example.com/v1` (production), `https://sandbox.platform.example.com/v1`
- **Format**: JSON only; `Content-Type: application/json`
- **Pagination**: Cursor-free offset pagination with `page`, `limit`, returning `meta.total`, `meta.pages`
- **Sorting**: `sort=field:asc|desc` query parameter
- **Timestamps**: ISO 8601 UTC (`2026-11-11T12:00:00Z`)
- **Monetary values**: Integer minor units (e.g. `150000` = BDT 1,500.00)
- **IDs**: BigInt; returned as JSON number; clients must use 64-bit integer support
- **Error format**: `{ code, message, details[], timestamp }`
- **Idempotency**: POST requests accept `Idempotency-Key` header; 24-hour replay window
- **Versioning**: URI path versioning (`/v1/`); breaking changes increment major version

HTTP Status	Meaning
200	Success (GET, PUT, PATCH)
201	Created (POST)
204	No Content (DELETE)
400	Bad Request / Validation error
401	Unauthorized (missing/invalid token)
403	Forbidden (insufficient permissions)
404	Resource Not Found
409	Conflict (e.g. seat already held)
422	Unprocessable Entity (business rule violation)
429	Too Many Requests
500	Internal Server Error

10. Implementation Status

✅ **Completed:** Full Prisma schema v2.5 (86+ models), database migrations, seed data, generated Prisma Client for all 86 models.

🚧 **In Progress:** NestJS modules and controllers. Currently only `AppModule`, `DatabaseModule`, and `AppController` are implemented. All 8 functional cluster modules remain to be built.

Cluster	Schema	Module	Controllers	Services
A — Tenancy & Access	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
B — Customers & Integration	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
C — Registries & Rules	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
D — Catalog	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
E — Operations & Allocation	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
F — Booking & Money	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
G — Supplier Integration	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending
H — Crew Management	✅ Complete	⬜ Pending	⬜ Pending	⬜ Pending

11. Revision History

Version	Date	Author	Summary of Changes
`v1.0`	2026-02-01	Platform Team	Initial SRS — 8 clusters, 78 API operations, DB schema v1.0 (72 tables)
`v1.1`	2026-05-10	Platform Team	Added DB schema v2.0 — ResourceDocument, ResourcePart, KycSubmission
`v1.2`	2026-08-15	Platform Team	Added Workflow engine models, LoyaltyAccount, PromoRedemption
`v2.0`	11 November 2026	Platform Team	Full sync with Prisma schema v2.5 (86 models), implementation status added, all 8 clusters fully detailed, ResourceCrew / CrewMemberRole / CrewRequirement documented, non-functional requirements formalized, security hardening section expanded